Documento de Arquitetura e Requisitos MVP: Sistema de Ponto Assíncrono

1. Visão Arquitetural e Decisões Técnicas

Este documento define a base técnica para o MVP do Sistema de Ponto. A arquitetura foi desenhada com foco em Alta Disponibilidade (AP), tolerância a falhas e interoperabilidade, adotando o modelo de Arquitetura Orientada a Eventos (EDA).

- Agnóstico de Stack (Interoperabilidade): A comunicação entre os serviços é feita de forma neutra, permitindo que produtores e consumidores sejam escritos em qualquer linguagem de programação.
- Broker de Mensageria: ActiveMQ via protocolo AMQP 1.0.

2. Requisitos Funcionais (RF)

- RF1 - Recepcionar Batida de Ponto (Producer): A API deve receber a matrícula (ID) do funcionário e enviar para a Fila 1.
- RF2 - Persistir Dados (Consumer A): Um worker lê a Fila 1, valida e salva no banco de dados.
- RF3 - Acionar Notificação: Após persistir, o Consumer A emite um evento para a Fila 2.
- RF4 - Comprovar Ponto (Consumer B): Lê a Fila 2 e simula envio de e-mail.
- RF5 - Consultar Histórico: API GET simples para listar os pontos batidos.

3. Diretrizes de Arquitetura de Software (Ports and Adapters)

O domínio da aplicação deve se comunicar com o mundo externo estritamente através de interfaces (Portas). A implementação real da tecnologia ficará na camada de infraestrutura (Adapters). A comunicação com filas deve usar o padrão Observer.

4. Estrutura do Repositório (Monorepo)

O projeto adota Monorepo para centralizar contratos e facilitar a execução local. A infraestrutura (Docker) fica na pasta /infra, isolada das pastas de cada aplicação (/api-producer, /worker-persistence, /worker-notification).

5. Estratégia de Configuração e Ambientes (12-Factor App)

Configurações que variam entre ambientes são injetadas exclusivamente via Variáveis de Ambiente (ENVs), priorizando a Nuvem em PROD/QA e arquivos .env isolados na máquina

local em DEV. A aplicação adota a política de "Fail Fast" para ausência de variáveis vitais em produção.

6. Modelagem de Dados (Relacional)

Para manter a simplicidade do ambiente open-source, o MVP utilizará um banco de dados relacional (H2 em memória para DEV, PostgreSQL via Docker para testes/produção).

Tabela: funcionario (Apenas Leitura no MVP)

```
CREATE TABLE funcionario (
  id VARCHAR(50) PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL
);
```

Tabela: registro_ponto (Append-only)

```
CREATE TABLE registro_ponto (
  id UUID PRIMARY KEY,
  mensagem_id UUID NOT NULL UNIQUE, -- ID da fila (Idempotência)
  id_funcionario VARCHAR(50) NOT NULL,
  timestamp_registro TIMESTAMP WITH TIME ZONE NOT NULL,
  criado_em TIMESTAMP WITH TIME ZONE DEFAULT
CURRENT_TIMESTAMP,
  CONSTRAINT fk_funcionario FOREIGN KEY (id_funcionario)
REFERENCES funcionario(id)
);
```

---

7. Topologia das Filas (Mensageria)

O fluxo principal utiliza duas filas no ActiveMQ. Para esta primeira iteração do MVP, o tratamento de falhas focará na rejeição natural (NACK) com retorno para a fila principal, sem a complexidade de Dead Letter Queues (DLQ).

- Fila 1 (Persistência): fila.ponto.processamento
  Propósito: Receber o comando da API. O Consumer A (Worker Persistence) consome desta fila.
- Fila 2 (Notificação): fila.ponto.notificacao
  Propósito: Receber o evento enriquecido do Consumer A. O Consumer B (Worker Notification) consome desta fila.